Information Processing and Management 62 (2025) 104042



Contents lists available at ScienceDirect

Information Processing and Management

journal homepage: www.elsevier.com/locate/ipm





OntoGenix: Leveraging Large Language Models for enhanced ontology engineering from datasets

Mikel Val-Calvo [a], Mikel Egaña Aranguren [b], Juan Mulero-Hernández [a], Ginés Almagro-Hernández [a], Prashant Deshmukh [c], José Antonio Bernabé-Díaz [c], Paola Espinoza-Arias [c], José Luis Sánchez-Fernández [c], Juergen Mueller [c], Jesualdo Tomás Fernández-Breis [a,*]

[a] Departamento de Informática y Sistemas, Universidad de Murcia, CEIR Campus Mare Nostrum, IMIB-Pascual Parrilla, Campus de Espinardo, Murcia, 30100, Spain
[b] University of the Basque Country (UPV/EHU), Bilbao, 48013, Spain
[c] BASF SE, Germany

ARTICLE INFO

Keywords:
Knowledge graphs
Large Language Models
Ontology engineering

ABSTRACT

Knowledge Graphs integrate data from multiple, heterogeneous sources, using ontologies to facilitate data interoperability. Ontology development is a resource-consuming task that requires the collaborative work of domain experts and ontology engineers. Therefore, companies invest considerable resources in order to generate and maintain Enterprise Knowledge Graphs and ontologies from large and complex datasets, most of which can be unfamiliar for ontology engineers. In this work, we study the use of Large Language Models to aid in the development of ontologies from datasets, ultimately increasing the automation of the generation of ontology-based Knowledge Graphs. As a result we have developed a structured workflow that leverages Large Language Models to enhance ontology engineering through data pre-processing, ontology planning, building, and entity improvement. Our method is also able to generate mappings and RDF data, but in this work we focus on the ontologies. The pipeline has been implemented in the OntoGenix tool. In this work we show the results of the application of OntoGenix to six datasets related to commercial activities. The findings indicate that the ontologies produced exhibit patterns of coherent modeling, and features that closely resemble those created by humans, although the most complex situations are better reflected by the ontologies developed by humans.

1. Introduction

Data management and exploitation in large organizations are complex tasks given the number and diverstiy of data sources and systems usually found in such organizations. In the last years, many of such companies have implemented Knowledge Graphs (KGs) as a technological solution for data integration, linking and exploitation, as it can be inferred by the participants in recent international KG conferences. The advent of KGs can be illustrated by the different domains in which they have been applied in the last years, such as recommendation systems (Xu, Li, Yang, Chen, & Yue, 2024), tourism (Gao et al., 2024), medicine (Shen, Pan, Xu, Gan, & An, 2023) or cultural heritage (Fan & Wang, 2022).

* Corresponding author.
E-mail address: jfernand@um.es (J.T. Fernández-Breis).

https://doi.org/10.1016/j.ipm.2024.104042
Received 13 June 2024; Received in revised form 4 September 2024; Accepted 28 October 2024
Available online 23 December 2024
0306-4573/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

KGs are graphs of data in which the nodes represent entities of interest, and edges represent relations between entities (Hogan et al., 2021). KGs generally combine data from different sources, and the structure and granularity of the resulting graphs may vary significantly, especially when the process for generating the graph does not apply a common schema. Ontologies can be used to provide such schema in KGs, that is, the meaning of entities and relations comes from classes and properties of existing ontologies. A rigorous and axiomatically rich ontology also enables computer systems to understand, and use data and knowledge effectively. Therefore, ontology engineering for KG development is not merely a technical task; it is a fundamental activity underpinning the global community's ability to manage data, so that data becomes accessible and useful now and in the future. In most cases, ontologies are manually built by a group of domain experts and ontology engineers, frequently in a complex, time and resources consuming process. Since such cost can be prohibitive for an organization, and limit the development of Enterprise KGs (EKGs) (Gomez-Perez, Pan, Vetere, & Wu, 2017), knowledge intensive, data-driven organizations need support in the development of ontologies and knowledge graphs.

Large Language Models (LLMs) are large-scale statistical models that have become popular due to their efficient processing and capability for generating large-scale text data. The potential of LLMs for increasing the productivity of different types of tasks has been recently studied in different areas like programming (Li, Su, Chen, Li, & Zhang, 2024; Weber, Brandmaier, Schmidt, & Mayer, 2024), plant label transcription (Weaver, Ruhfel, Lough, & Smith, 2023), or academic work and education (Lonsdale, O'Reilly-Shah, Padiyath, & Simpao, 2024; Noy & Zhang, 2023). Therefore, our hypothesis is that LLMs can help in automating the development of ontologies.

In this work, we explore the incorporation of advanced LLMs (Brants, Popat, Xu, Och, & Dean, 2007) to the ontology development process. It should be noted that, in organizational settings, ontologies must be generated for knowledge areas unfamiliar for the ontology engineers. Hence, our goal is not full automation but generating knowledge about LLMs performance and potential support for drafting ontologies for those unfamiliar datasets. Besides increasing the automation of the development of ontologies, LLMs may help to reduce inaccuracies and inconsistencies inherent to manual methods, ensuring the creation of more reliable and robust ontologies. Automating the development of ontologies also contributes to the scalability of the generation of KGs. This also applies to the mappings necessary in the conversion of data to KGs, since those mappings should be built by applying those ontologies.

In this study, we propose a workflow through which different instruction sets are provided to the LLM for planning and building the ontology based on the input dataset. This workflow has been implemented in the OntoGenix software.[1] In order to facilitate the use of OntoGenix, a GUI has been developed. The workflow has been evaluated by applying OntoGenix to publicly available datasets related to products, sales or customer satisfaction. In the experiments, we will compare the ontologies and mappings developed by OntoGenix with the ones developed by ontology engineers and domain experts.

We believe that this work generates insights about how LLMs can contribute to improving ontology and data engineering, and that approaches like ours contribute to their scalability. The contributions of our work are (1) a modular and configurable workflow (see Section 3.1) allowing for the customization, and scaling of ontology and data engineering processes to meet specific domain requirements and adapt to evolving domain landscapes; and (2) the OntoGenix software tool that implements the workflow, which is freely available (see Section 4.1).

2. State of the art

Recent advancements in ontology engineering and KG creation have been greatly influenced by advanced computational methods and technologies. The forefront of this field is characterized by an increasing trend towards automation (Hatakeyama-Sato, Yamane, Igarashi, Nabae, & Hayakawa, 2023; Mateiu & Groza, 2023), especially evident in generating ontologies from tabular data, since this is a structure familiar for domain experts. Many datasets are made available in formats such as csv, tsv, or xlsx. For example, most datasets in machine learning oriented portals such as Kaggle are in such formats. Key developments include the integration of Semantic Table Interpretation techniques, as outlined by Liu et al. (2023). These techniques are vital for extracting meaningful information from tabular data, and establishing connections between the rows and columns of the tabular structure and the entities, concepts, or relations within KGs. This area of research is further expanded in studies focusing on Column Linking or Column Type Annotation for previously unseen ontologies (Dash, Bagchi, Mihindukulasooriya, & Gliozzo, 2023), emphasizing the need for adaptable tools capable of handling various data formats and domain-specific intricacies.

TANGO (Table ANalysis for Generating Ontologies) (Tijerino, Embley, Lonsdale, Ding, & Nagy, 2005) addresses the challenges in semantics, mutual understanding, concept matching, and interoperability inherent in the information-explosion era. It was seen at that time as a means to facilitate the realization of the Semantic Web by automating the process of creating ontologies from real-world knowledge. The creation of tools such as MOGO (Mini-Ontology GeneratOr) (Lynn, 2008) and Populous (Jupp et al., 2012) was a significant advance in facilitating ontology creation. MOGO proposed the generation of mini-ontologies from a canonicalized table structure, suited for any set of tabular data, while Populous offered a user-friendly interface for domain experts, enabling them to contribute to ontology development without the complexity of ontology languages. This emphasis on user-centric tools demonstrated an awareness of the necessity to bridge the gap between technical ontology development and domain-specific input.

The need for scalable solutions that can adapt to the complexity of modern data integration and management is evident (Mizoguchi & Bourdeau, 2000). The current research path in ontology engineering and KG creation points towards more

[1] https://doi.org/10.5281/zenodo.13589880

automated, efficient, and user-friendly methodologies. These developments cater to both technical experts and domain-specific users, contributing to the diversity of semantically rich ontologies and KGs.

LLMs are large-scale statistical models that have supported various tasks in recent years (Chung, Cahyawijaya, Wilie, Lovenia, & Fung, 2023; Demszky et al., 2023; Kannan, Venkatesh, & Min, 2023; Thoppilan et al., 2022; Zhou, Song, Yao, Shu, & Ma, 2024), including those related to ontology engineering like Knowledge Base population or KG construction (Caufield et al., 2024; Fan & Chen, 2024), ontology learning (Babaei Giglou, D'Souza, & Auer, 2023a), and sentence transformation into OWL (Web Ontology Language) content (Mateiu & Groza, 2023). The use of LLMs in managing extensive input contexts, facilitating visual programming interactions, and performing step-aware verification for reasoning tasks (Cai et al., 2023; Li et al., 2023; Liu, Lin et al., 2024) highlights the evolving role of LLMs in ontology engineering, showcasing their potential to enhance productivity.

Those efforts offer insights into optimizing the use of LLMs, such as GPT-4, for tasks requiring deep contextual understanding and complex instruction following, indicating ongoing improvements in ontology engineering and KG creation. The combination of logic programming with LLMs (Madani & Joseph, 2023), as demonstrated by methods merging Prolog with knowledge-based question answering, suggests a promising direction for augmenting the reasoning capabilities of LLMs in KG contexts (Zulkipli, Maskat, & Teo, 2022). Prompt engineering techniques like "chain-of-thought" (Wei et al., 2022) and "chain-of-memory" (Hu et al., 2023) enhance multi-step problem solving and complex data operations by incorporating databases as external symbolic memory into LLMs (Al-Aswadi, Chan, & Gan, 2020).

Finally, we can highlight some recent works that use LLMs for increasing the automation of ontology and KG generation. An approach for enriching ontologies by translating text into Description Logic (DL) has been presented in Mateiu and Groza (2023). This approach is based on performing a specific fine-tuning of the LLM. The training set comprised prompts including natural language sentences with their corresponding DL axioms. More recently, Ontogenia (Lippolis, Ceriani, Zuppiroli, & Nuzzolese, 0000) proposed a method for ontology generation based on metacognitive prompting using LLMs. The process is driven by competency questions. The LLM is expected to understand the competency questions , and to generate an ontology capable of satisfying them. The results were evaluated with a dataset related to WildLife. The results were promising in terms of the potential of LLMs for supporting the ontology generation process. Finally, the use of LLMs for ontology learning (Babaei Giglou, D'Souza, & Auer, 2023b) is also relevant for this work, since the researched aspects included the capability of LLMs to recognize taxonomic and non-taxonomic relations, highlighting the potential and limitations of LLMs for sophisticated modeling.

3. Materials and methods

3.1. The workflow for generating the ontology

We have designed an elaborate workflow for enabling the generation of an ontology from a structured data file. The architecture has been designed to promote advanced ontology and data engineering by incorporating LLMs (see Fig. 1). The figure depicts the modular workflow from input to output, encompassing the key components such as user interaction (A), actions management (B), data preprocessing (C), schema definition (D), ontology creation (E), mapping and data generation (F). The integration of LLM capabilities is central to the process, enhancing the ontology engineering and data conversion processes. The workflow is designed for interactive user engagement, with mechanisms for self-repairing mappings to ensure the accuracy and relevance of the ontologies and KGs generated.

3.1.1. (A) User interaction

The User Interaction module (Fig. 1A) allows the user to iterate through the design of the ontology using natural language, while also receiving feedback. The user can load the source CSV data file for which the following artifacts will be created: ontology, mapping, and the transformation into the final graph. Upon loading the data, a pre-processing step generates a statistical summary of the content. Similarly, the user can interactively define the structure of the ontology to be designed.

3.1.2. (B) Actions manager

The Actions Manager (Fig. 1B) plays a critical role in orchestrating the workflow. It manages the state and determines subsequent actions, using the Assist Bot to guide the process based on the current GUI state and user interactions. The Actions Manager serves as a crucial guide and facilitator for users, especially those new to ontology design, navigating the ontology construction process within the GUI framework. The Actions Manager receives user input and processes the requests accordingly.

The Actions Management module is specifically designed to empower users with little to no previous experience in ontology design. It makes the complex process of designing ontologies more accessible and manageable. By providing contextual assistance and managing the flow of actions, the module significantly enhances the user experience, making the intricate task of ontology creation more user-friendly and intuitive.

The **Assist-bot** agent guides the user interaction with messages that show the steps the user can take at each stage of the process. To provide this supervision and help, the agent records the development of the ontology generation process, mapping, and the creation of the graph as a result. The Assist Bot plays a key role in providing tailored instructions to users. This agent is responsible for interpreting user prompts. It decides which of the other modules should receive a given prompt, making this decision based on both the current state of the ontology development process and the historical context of previous actions. The Assist Bot ensures a smooth flow of the ontology construction process, and it intelligently performs the next action, directing user inputs to the appropriate modules at the right stages.

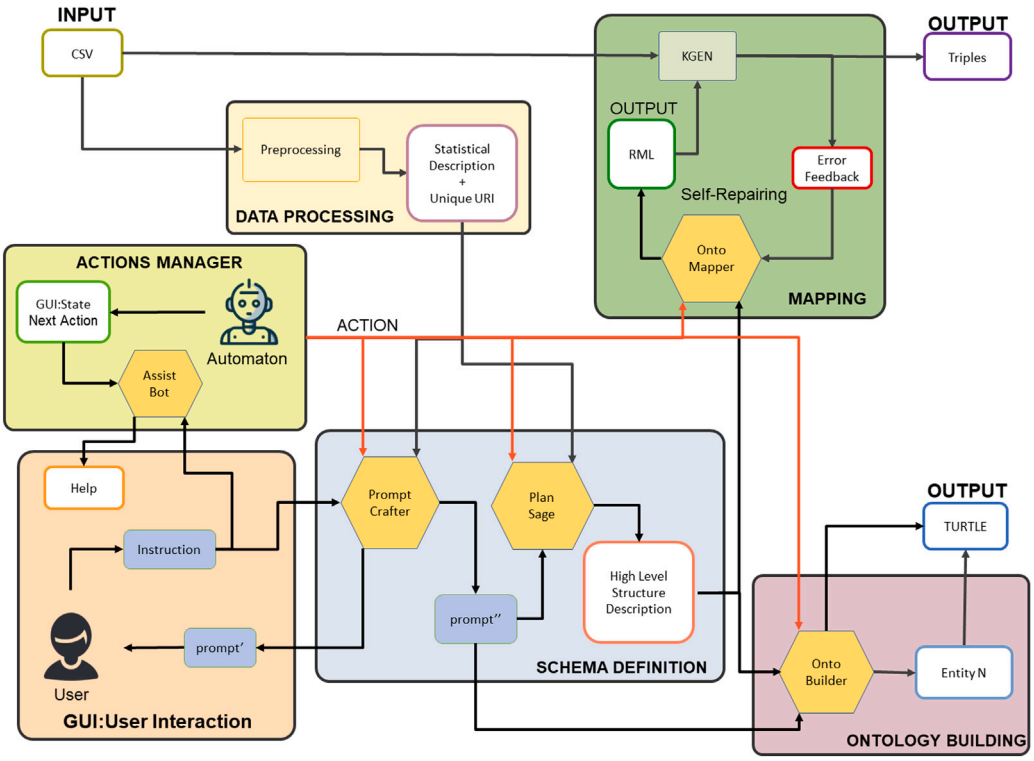


Fig. 1. The Architecture Diagram. This figure illustrates the workflow for enhanced ontology engineering using LLMs. It details the flow from data input, through the various stages of processing, actions management, schema definition, to the final output in ontology format. The system is composed of the following key modules: **(A) GUI: User Interaction:** This module represents the user's interaction with the system. It includes a help interface and allows the user to input prompts, which are then processed by the Actions Manager. **(B) Actions Manager:** This module manages the state and actions of the system. It includes the Assist Bot, which assists in determining the next actions based on the GUI state, and interacts closely with the user via the Automaton. **(C) Data Processing:** This stage involves preprocessing the input data (e.g., CSV files), generating a statistical description, and assigning unique URIs for further processing. **(D) Schema Definition:** This module includes the Prompt Crafter and Plan Sage, which work together to define the high-level structure of the ontology. The output from this module is a detailed prompt that guides the ontology building process. **(E) Ontology Building:** This module processes the high-level structure description to generate the ontology and mapping to generate the ontology in the desired formats (e.g., TURTLE and entity *N* triples). **(F) Mapping:** This module handles the self-repairing mechanism and error feedback. It includes Onto Mapper, which converts the high-level structure into a mapping format (RML) and generates the RDF data.

3.1.3. (C) Data processing

Data processing begins with the module depicted in Fig. 1C, where input data, such as CSV files, undergo preprocessing. This module also generates statistical descriptions and assigns unique URIs, which are essential for subsequent stages in the ontology generation process. The data preprocessing step is pivotal in ensuring the data is suitably structured and annotated for the effective generation of ontologies. These are the activities carried out in this step:

- Reading and initial processing of the dataset: The dataset is first read from a CSV file, ensuring that the encoding and memory considerations are appropriately handled.
- Unique Identifier Determination: Generation of a unique identifier for each record of the dataset, crucial for distinguishing individual records.
- Generation of Unique and Readable URIs: For each record, a Unique and Readable URI is generated, using a combination of selected column values and a hashed unique identifier. This step ensures that each record has a distinct, machine-friendly identifier.
- Summary of column data: The script categorizes and summarizes columns based on their data type – numerical, textual, or categorical – providing essential statistical insights into each type.
- Merging of summaries: The individual summaries are then merged to form a comprehensive overview of the dataset, highlighting the key characteristics of each column type.
- Cleaning and finalizing the summary: Any non-informative or placeholder entries are cleaned from the summary, ensuring clarity and relevance.
- Conversion to JSON Format: The summarized data is converted into a readable JSON format for ease of use, which can be optionally saved to a file.

3.1.4. (D) Schema definition

As shown in Fig. 1D, the Schema Definition module is central to defining the high-level structure of the ontology. Its components, namely, the Prompt Crafter and the Plan Sage, collaboratively refine that high-level structure into a detailed prompt that guides the ontology building phase. The Prompt Crafter and the Plan Sage agents leverage the capabilities of advanced LLMs to assist in defining high-level structural descriptions for the ontology.

The Prompt Crafter. This agent analyzes and constructively criticizes the design prompt introduced by the user, returning an improved version of the prompt in a human-agent loop until the human considers the prompt to be sufficiently refined. The **Prompt Crafter** agent employs a sophisticated prompt mechanism to facilitate the creation of ontologies as it begins with a prompt that invites ChatGPT, acting as an Expert Ontology Engineer, to assist in crafting a prompt based on the provided JSON data. This process includes the following steps:

1. **Role Assignment:** ChatGPT is designated as an Expert Ontology Engineer.
2. **Objective:** Aid in developing ontology-structuring prompts from JSON data.
3. **Inputs:**
   (a) *Prompt:* Initial user-provided prompt.
   (b) *Data:* JSON data for ontology basis.
   (c) *Task:* Guidance for crafting an effective prompt.
4. **Refinement Process:**
   (a) *Sections:* Creation of prompt, critique, and clarifying questions.
   (b) *Iteration:* User feedback and ChatGPT adjustments until prompt finalization.
5. **User Perspective:** Ensure that the prompt matches the user's intent.
6. **Inspiration:** Example provided as reference.
7. **Engagement:** Creative and thoughtful approach in prompt development.

Next, the Prompt Crafter does an iterative refinement of the prompt based on user feedback and additional information. This process ensures that the prompt is tailored to the user's needs and the specific context of the ontology. An illustrative example of a prompt generated by the Prompt Crafter might begin with "As an expert ontology engineer, help me describe the nature of the provided data". The integration of the Prompt Crafter guarantees that the ontology classes, subclasses, properties, and relationships are not only accurately defined but also aligned with the user's specific requirements and communication style. This ensures logical coherence and efficient modeling of the ontology at the level of personalization that is required.

Plan Sage. The **Plan Sage** agent employs a comprehensive prompt designed to analyze and synthesize JSON data for ontology development. This module begins with a prompt inviting the LLM (acting as an expert ontology engineer) to analyze a provided JSON dataset. It takes the improved prompt from the Prompt Crafter agent, and generates a high-level definition of the proposed ontology structure, having considered both the statistical summary generated from the data and the user's design prompt. The outcome can be modified by the user. This result is key to the rest of the process, as this high-level definition is shared by the agents responsible for generating the ontology (OntoBuilder) and the mapping (OntoMapper). The prompt looks like this:

1. **Objective**: Synthesize JSON data for ontology development.
2. **Input Specification**:
   - JSON Data: Input in JSON format.
   - Task Description: Task defined related to JSON data.
3. **Data Analysis**:
   (a) Critical Analysis: Perform a critical review of JSON data.
   (b) Synthesized Description: Produce a comprehensive summary of the data.
   (c) Column Relations: Detail potential relationships for ontology classification.
4. **Structured Summary Components**:
   (a) Prefix: Use "https://vocab.um.es#" for OWL types if no prefix is provided.
   (b) Data Description: Detail each JSON data column.
   (c) Classes and Subclasses: Enumerate and describe relationships.
   (d) Properties: List object and data type properties with domain and range.
   (e) External Links: Suggest connections to external ontological resources.
5. **Restrictions**: Exclude additional descriptions, introductions, explanations, and notes from the end of the document.

3.1.5. (E) Ontology building

The output of the workflow is produced by the Ontology Building module E (Fig. 1). This module uses the structure provided by earlier stages to generate the ontology.

Through the **OntoBuilder** agent, the high-level ontology definition is translated into a Turtle file representing the ontology. Improvements to the definition of the ontology entities can be requested at this stage with the help of the AssistBot and Prompt Crafter agents. This means there are two methods for designing the ontology: one at a global level through the Plan Sage agent, and another one at an entity level through the OntoBuilder agent. OntoBuilder is responsible for defining classes, subclasses, properties, instances, and their relationships in Turtle format, ensuring the alignment of the content of the ontology with the conceptual model and the semantic content of the data. The OntoBuilder agent uses two distinct prompts to facilitate ontology creation and entity enrichment, namely, the ontology building prompt, and the entity enrichment prompt, which are described next.

Ontology building prompt. The agent starts with a prompt requesting the generation of an ontology in TURTLE syntax from provided JSON data and a schema description.

1. **Objective:** Create an ontology in TURTLE syntax from JSON data and schema.
2. **Input Details:**
    (a) *JSON Data:* The main source in JSON format.
    (b) *Data Description:* Context for the JSON data.
    (c) *Adherence:* Follow instructions precisely.
3. **Ontology Creation:**
    (a) *Completeness:* Ensure the TURTLE content is comprehensive.
    (b) *Structure Focus:* Prioritize the logical ontology structure.
    (c) *Class Interconnection:* Connect all classes as per schema.
    (d) *Properties:* Define domain and range for each property.
    (e) *Omissions:* Exclude labels, comments, descriptions, instances.
4. **Writing Restrictions:**
    (a) *No Introductions:* Avoid introductions or explanations at the end.
    (b) *No Notes:* Do not include concluding notes.

This approach ensures a comprehensive and detailed ontology structure, focusing on the essential elements without strange information.

Entity enrichment prompt. The entity enrichment process entails an in-depth examination of a specific ontology entity, a detailed analysis of the data description, and a comprehensive investigation of the rationale behind the ontology's design.

1. **Role:** ChatGPT as expert ontology engineer.
2. **Objective:** Enhance a specific ontology entity.
3. **Inputs:**
    (a) *Ontology:* The complete ontology.
    (b) *Data:* Data description for ontology.
    (c) *Insights:* Rationale for ontology design.
4. **Improvement Task:**
    (a) *Analysis:* Review ontology, data, and insights.
    (b) *Entity Focus:* Target improvements for an entity.
    (c) *Enhancements:* Suggest entity refinements.
5. **Output:**
    (a) *Syntax:* Present enhanced entity in TURTLE.
    (b) *Entity-Only:* Include only the entity's TURTLE content.
6. **Style and Restrictions:**
    (a) *Conciseness:* Exclude introductions, explanations, and notes.

This step ensures that each ontology entity is optimally defined and aligns with the overall objectives and data characteristics.

By integrating these prompts into the OntoBuilder module, our approach facilitates a structured and logical development of the ontology. This process ensures that each aspect of the ontology, from the high-level structure to individual entities, is carefully crafted and aligned with the overarching goals and data context.

3.1.6. (F) Mapping and KG generation

The Mapping module (Fig. 1F) is integral to the system's self-repairing capability. It includes OntoMapper, which transforms the high-level structure into a mapping format, ensuring that the ontology generation is accurate and consistent with the input data. The Mapping module integrates the capabilities of KGEN (Knowledge Graph ENgine), RDF Mapping Language (RML), and self-repairing mechanisms. This combination allows for the efficient conversion of structured data into a KG, enhancing the accuracy of mappings through error feedback processes.

OntoMapper. The **OntoMapper** agent employs two specific prompts for RML generation and error correction. Its main role is to generate the mapping between the ontology definition and the source dataset. It takes the high-level structure definition of the ontology as input and generates a mapping in RML. This agent uses a module called KGEN that obtains the mapping and executes the transformation of the source data into a graph using the mapping. The errors of the process are captured and sent back to the agent to modify the mapping. The final result is an RDF (Resource Description Framework) file representing the graph. The prompt looks like this:

1. **Role:** ChatGPT as ontology translation specialist.
2. **Objective:** Generate RML mappings from an existing ontology.
3. **Inputs:**
    (a) *Rationale:* Explanation for the ontology's data source and structure.
4. **Output Guidelines:**
    (a) *URIs:* Construct unique URIs with 'FAIR_URI' and entity names.
    (b) *Prefixes:* Start RML with necessary prefixes, referencing them in triple blocks.
    (c) *Data Source:* Map from a CSV file, presented in Turtle syntax for clarity.
    (d) *URI Use:* Employ base prefix for URIs, not generic examples.
5. **Content Restrictions:**
    (a) *Completeness:* Provide full RML mapping definitions.
    (b) *Conciseness:* Exclude generic notes and focus exclusively on RML code.

For error correction, the OntoMapper uses the following **self-repairing prompt**:

1. **Task:** Produce RML mapping, rectifying a given error.
2. **Error Resolution:** Directly correct the identified error.
3. **Inputs:**
    (a) *Error:* Description of the mapping error.
    (b) *Rationale:* Background of the JSON data and ontology structure.
4. **Output Requirements:**
    (a) *URIs:* Create unique URIs with 'FAIR_URI' and entity names.
    (b) *Prefixes:* Include and reference necessary prefixes in RML.
    (c) *CSV Reference:* Use CSV, not JSON, in RML mappings.
    (d) *RML Class Mapping:* Adhere to the specified RML class mapping format.
    (e) *URI Usage:* Utilize base prefix for URIs.
5. **Style Guidelines:**
    (a) *RML Focus:* Deliver pure RML code, excluding explanations or notes.
    (b) *Precision:* Ensure complete and accurate mappings for each class.
    (c) *Prefix Attention:* Carefully include all RML prefixes.

The **Knowledge Graph Generation** (KGen) module works in tandem with the OntoMapper, executing commands to generate the KG. The process includes (1) serializing the graph in N-Triples format; and (2) providing feedback on the generation process, including error tracing.

Table 1
Description of the selected datasets.

Dataset	Description	Rows	Columns
Airlines Customer Satisfaction	Details of customers who have already flown with them, including the feedback of the customers on various contexts (https://labur.eus/SN5n1)	129 880	23
Amazon Ratings	Customer reviews and ratings of beauty related products sold on their website (https://labur.eus/voslX)	2 023 070	4
BigBasket Products	Products listed on the website of online grocery store Big Basket (https://labur.eus/tnqzt)	8208	9
Brazilian E-commerce	Brazilian e-commerce public dataset of orders made at Olist Store (https://labur.eus/WM3cS)	99 441	5
Customer Complaint	Collection of complaints about consumer financial products (https://labur.eus/NlmH7)	745 392	18
E-commerce	Transactions for a UK-based and registered non-store online retail (https://labur.eus/AJ8bF)	541 909	8

The **RAG OntoMapper** orchestrates the overall mapping and KG generation process. It handles (1) building the KGen with dataset paths and configuration; (2) saving the RML mapping generated by OntoMapper; and (3) running the KGen to materialize the KG, capturing any errors.

In the final stages, iterative refinement is allowed. The process actively engages domain experts and ontology engineers in a feedback loop to test the results and ensure its accuracy and applicability.

3.1.7. Summary of the process

Once described the modules, we summarize the process for the generation of the ontology. The flow and the interaction are driven by the Actions Manager module, where the Assist Bot uses function calling to coordinate the process. First, the user must load a structured data file (CSV format). The Data Processing module generates a statistical description of the data in order to facilitate the ingestion of the data to the Plan Sage agent. No matter the size of the original data source, the statistical description of the data, which enables to work with a structured, standardized and synthesized format. The Prompt Crafter will provide an initial prompt to guide the Plan Sage in the ontology definition, and the user can iterate as needed to align its needs. This process concludes by feeding the Plan Sage agent with a clear and well-suited prompt for the use case. The Plan Sage agent will generate a high-level description of the ontology, which can be refined manually by the user. Once this process is done, the user can ask the Assist Bot to generate the ontology and its associated KG. Furthermore, the Assist Bot can assist the user to define an alternative prompt to ask the OntoBuilder to provide specific definitions into low level entities from the defined ontology.

3.2. Experimental evaluation method

We have evaluated our method using six Kaggle datasets related to commercial activities of organizations. For each dataset we have developed two ontologies: one ontology was generated by applying the LLM-supported workflow implemented in the OntoGenix tool (see Section 4.1), and the other one was developed by a human ontology engineer. Two humans (A and B) played the role of ontology engineer, each of them manually developing three ontologies. Both humans had an experience of at least 5 years in the development of ontologies from different domains. The development of both types of ontologies was independent, and the people involved in both processes was different.

3.2.1. Datasets

Table 1 describes the datasets that have been included in the experimental evaluation performed. They are datasets freely available at Kaggle, all of them related to commercial activities of organizations.

3.2.2. Assessment of the quality of the ontologies

The evaluation encompassed both a quantitative and a qualitative comparative assessment of the human-generated and the ones generated with the support of the LLM. The quantitative assessment was facilitated by the utilization of automated ontology quality evaluation tools:

- OQuaRE (Duque-Ramos, Fernández-Breis, Stevens, & Aussenac-Gilles, 2011) presents 19 quality metrics[2] (see Table 2). We have calculated and compared the values of the metrics for the ontologies. OQuaRE generates two types of values for a metric. The first value is the raw value, whose range depends on the nature of each metric. The second value is the scaled value or quality score, which is a discrete value in the range 1–5, where 1 is the lowest score, and 5 is the highest one. It should be noted that not always a high raw value of a metric is desired from a quality perspective. Therefore, higher raw values of metrics may be

[2] https://semantics.inf.um.es/oquare/metrics.html

Table 2
Description of the 19 OQuaRE metrics applied in this work: Column 1 shows the acronym of the metric and its reference, with the ontology entity targeted by the metric in brackets; Column 2 describes how the metric is calculated.

Metric	Description
ANOnto (Annotations)	Mean number of annotations properties per classes
AROnto (Attribute richness)	Number of restrictions of the ontology per classes
CBOnto (Coupling)	Number of direct ancestors of classes divided by the number of classes minus subclasses of thing
CBOnto2 (Coupling)	Mean number of direct ancestors per classes
CROnto (Individuals)	Mean number of individuals per classes
DITOnto (Depth of the hierarchy)	Length of the longest path from thing to a leaf classes
INROnto (Descendants)	Mean number of subclasses per classes
LCOMOnto (Cohesion)	Mean length of all paths from leaf classes to thing
NACOnto (Ancestors of leaf classes)	Mean number of superclasses per leaf classes
NOCOnto (Descendants)	Number of the direct subclasses divided by the number of classes minus the number of leaf classes
NOMOnto (Properties)	Mean number of object and data property usages per class
POnto (Ancestors)	Mean number of direct ancestors per class
PROnto (Property richness)	Number of subclass of relationships divided by the number of subclass of relationships and properties
RFCOnto (Properties usage)	Number of usages of object and data properties and superclasses divided by the number of classes
RROnto (Properties usage)	Number of usages of object and data properties and super classes divided by the number of classes
TMOnto (Multiple inheritance)	Mean number of classes with more than one ancestor
TMOnto2 (Multiple inheritance)	Mean number of direct ancestors of classes with more than 1 direct ancestor
WMCOnto (Complexity)	Mean length of the paths from thing to a leaf classes
WMCOnto2 (Complexity)	Mean number of paths from thing to a leaf classes

Table 3
Definition of 19 OOPS! pitfalls. Only those found in the ontologies generated in this work have been included. The full catalogue can be found at https://oops.linkeddata.es/catalogue.jsp.

Pitfall	Definition	Type
P02	Creating synonyms as classes	Minor
P04	Creating unconnected ontology elements	Minor
P07	Merging different concepts in the same class	Minor
P08	Missing annotations	Minor
P10	Missing disjointness	Important
P11	Missing domain or range in properties	Important
P13	Inverse relationships not explicitly declared	Minor
P19	Defining multiple domains or ranges in properties	Critical
P21	Using a miscellaneous class	Minor
P22	Using different naming conventions in the ontology	Minor
P24	Using recursive definitions	Important
P31	Defining wrong equivalent classes	Critical
P32	Several classes with the same label	Minor
P34	Untyped class	Important
P35	Untyped property	Important
P36	URI contains file extension	Minor
P37	Ontology not available on the web	Critical
P38	No OWL ontology declaration	Important
P39	Ambiguous namespace	Critical

associated with lower scaled ones. An example of such situation is TMOnto, which stands for the ratio of classes with multiple ancestors. This will be further detailed in Section 4.3.1.

- The Ontology Pitfall Scanner (OOPS!) (Poveda-Villalón, Gómez-Pérez, & Suárez-Figueroa, 2014) checks for the occurrence of 41 pitfalls in ontology modeling, classified as critical, important or minor in a catalogue.[3] We have identified and compared the pitfalls in the human and the ontologies generated with the support of the LLM. The pitfalls found in those ontologies are described in Table 3.

The qualitative assessment was done by a third ontology engineer with more than 10 years of experience in ontology development, by manually assessing the two ontologies developed for each dataset.

It should be noted that those datasets were not randomly selected: they were chosen because they provided some classes of interest. In this case, the classes of interest are classes included in a reference ontology that was provided to us by BASF SE. We have also analyzed how such classes have been represented in the ontologies generated with the support of the LLM.

3.2.3. Estimating effort saving

Our workflow aims at saving time in the development of the ontologies. As described in the previous subsection, for each dataset two ontologies were generated: one by applying the LLM-based workflow, namely, OntoGenix ontology, and the human ontology.

[3] https://oops.linkeddata.es/catalogue.jsp

Table 4
Main descriptive metrics of the ontologies obtained for each dataset: number of classes (C), object properties (OP), datatype properties (DP), annotation properties (AP), individuals (I) and logical axioms (Ax).

Dataset	LLM						Human					
	C	OP	DP	AP	I	Ax	C	OP	DP	AP	I	Ax
Airlines	32	8	7	7	30	72	7	6	2	2	25	41
Amazon	6	2	4	9	0	17	3	2	4	1	0	12
BigBasket	4	6	18	8	0	49	3	2	12	4	0	34
Brazilian	7	3	5	9	6	22	10	10	27	3	0	75
Customer	21	9	20	14	13	83	9	7	10	2	0	40
E-commerce	11	5	8	8	21	47	8	7	11	4	9	68

We have measured the time spent in (1) the development of the ontology from scratch, and (2) starting with the OntoGenix ontology and transforming it to the human one. The development from scratch requires the following activities: (1) analysis of the CSV data; and (2) construction of the ontology. The development from the OntoGenix ontologies requires the following activities: (1) analysis of the CSV; (2) revision of the ontology generated with OntoGenix, which includes generating the ontology with OntoGenix; and (3) development of the human ontology from the OntoGenix.

The analysis of the CSV data is done in both cases, so it will not be considered in this analysis. Therefore, the time from scratch will be the time spent by the human in the modeling of the ontology, and the time starting from the OntoGenix ontology will include the time of running and reviewing the OntoGenix result, and modifying that ontology. Once both times are available, the time saved in absolute and relative terms is calculated.

The reconstruction of the human ontology was done by the ontologists involved in the development of the human ontologies. Besides, a qualitative rating to the effort needed to transform the OntoGenix ontology into the human ontology was assigned in the range 1-5: 1 (very little effort), 2 (little effort), 3 (moderate effort), 4 (high effort), 5 (very high effort). This assessment was done by the ontologists involved in the development and evaluation of the human and the OntoGenix ontologies.

4. Results

This section describes the results of the work, which are publicly available on two GitHub repositories: (1) the repository where the OntoGenix software is hosted[4]; (2) the repository containing the ontologies generated by the human developers and OntoGenix for all the datasets, as well as the results of the evaluation of the quality of the ontologies.[5] OntoGenix is also available in Zenodo.[6]

4.1. The OntoGenix software

OntoGenix is a multi-agent meta-programming software designed to promote advanced ontology engineering by incorporating LLMs. The OntoGenix software provides a Python implementation of the workflow described in this work, and it currently uses GPT-4 (version gpt-4-1106-preview) as LLM.

OntoGenix has been designed using a Model-View-Controller paradigm, where the view is composed by the User Interaction module, the Controller is implemented in the Action Manager, and the Model includes Data Processing, Schema Definition, Ontology Building and Mapping and KG Generation. A user interface for executing the workflow has been implemented in Python, and a screen snapshot is shown in Fig. 2. On the left, the GUI provides an interactive prompt-based chat window for user guidance, and on the right, sections for data importation and ontology modeling are displayed, including CSV Inspector, Ontology (OWL) Mapping, and Diagram visualization tools. The central area showcases a prompt with user queries and critique, reflecting the system's capability to facilitate a dialog between the user and the software, thus streamlining the ontology development process.

4.2. Description of the ontologies generated

Table 4 shows the main descriptive metrics of the ontologies generated for each dataset: number of classes (C), object properties (OP), datatype properties (DP), annotation properties (AP), individuals (I) and logical axioms (Ax). We can see that the ontologies developed by OntoGenix tend to have more content. The number of entities in both human and OntoGenix ontologies vary with the dataset, which means that both tend to adapt the modeling to the actual datasets.

[4] https://github.com/tecnomod-um/OntoGenix
[5] https://github.com/tecnomod-um/OntoGenixEvaluation
[6] https://doi.org/10.5281/zenodo.13589880

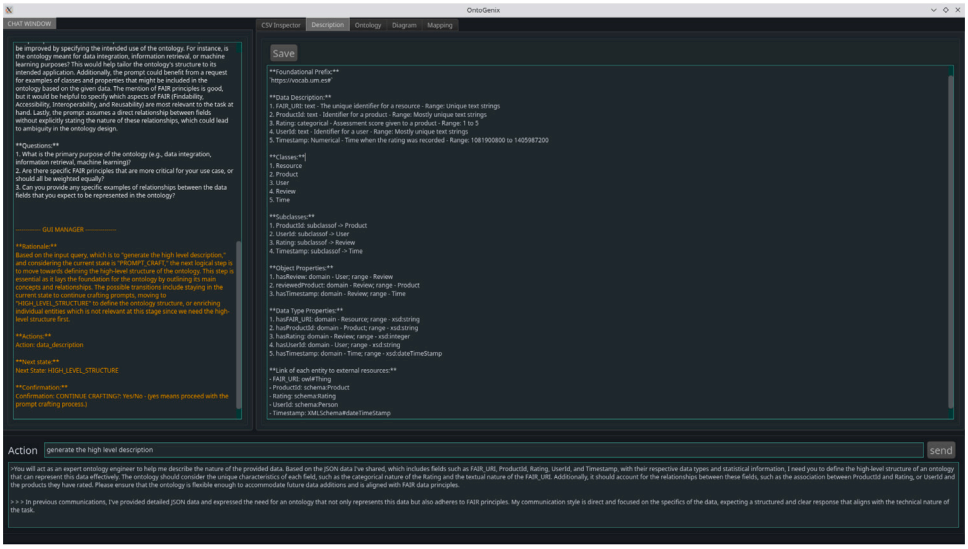


Fig. 2. The OntoGenix Interface showcasing the interaction workflow for ontology engineering assisted by a Large Language Model (LLM). The GUI is divided into various sections for inputting data, crafting prompts, and managing the ontology construction process. The left section displays a chat interface where the user communicates with the LLM, providing prompts and receiving guidance. The central section presents a CSV inspector and mapping tools for data structuring. The right section is reserved for visualizing the ontology through diagrams and logs, thus integrating the user feedback and the LLM recommendations into a cohesive ontology engineering environment.

Table 5
Raw values of the human (H) and OntoGenix (O) ontologies for the 19 OQuaRE metrics for the Airlines, Amazon and BigBasket datasets. The column Best means whether the highest or the lowest raw value of the metric means the highest quality score. Numbers in bold mean better quality values for the OntoGenix ontologies. The numbers of the Best value row stands for the number of metrics for which a better value was achieved by the human or OntoGenix ontology.

METRIC	Best	AIRLINES		AMAZON		BIG BASKET	
		H	O	H	O	H	O
ANOnto	Highest	0.88	**2.41**	0.75	0.00	0.25	**3.00**
AROnto	Highest	1.00	**6.97**	1.50	0.86	1.00	**2.80**
CBOOnto	Lowest	0.88	1.06	0.75	1.00	0.75	0.80
CBOOnto2	Lowest	0.88	1.06	0.75	1.00	0.75	0.80
CROnto	Highest	3.13	0.94	0.00	0.00	0.00	0.00
DITOnto	Lowest	2.00	4.00	2.00	3.00	2.00	2.00
INROnto	Highest	0.88	**1.06**	0.75	**1.00**	0.75	**0.80**
LCOMOnto	Lowest	1.00	2.73	1.00	1.75	1.00	1.00
NACOnto	Lowest	1.00	1.07	1.00	1.33	1.00	1.00
NOCOnto	Lowest	7.00	8.50	3.00	**1.75**	3.00	4.00
NOMOnto	Lowest	3.00	**1.63**	4.50	**2.86**	13.00	16.80
POnto	Highest	0.00	**0.63**	0.00	**0.43**	0.00	**0.40**
PROnto	Highest	0.77	0.60	0.86	0.74	0.95	**0.96**
RFCOnto	Lowest	3.88	**2.69**	5.25	**3.86**	13.75	17.60
RROnto	Highest	0.23	**0.40**	0.14	**0.26**	0.55	0.05
TMOnto	Lowest	0.00	0.00	0.00	0.17	0.00	0.00
TMOnto2	Highest	0.00	0.00	0.00	**2.00**	0.00	0.00
WMCOnto	Lowest	1.00	4.97	1.00	2.33	1.00	1.00
WMCOnto2	Highest	1.00	**1.82**	1.00	**1.33**	1.00	1.00
	Best value	**9**	**8**	**10**	**8**	**6**	**5**

4.3. Assessment of the ontologies

4.3.1. Quantitative assessment with OQuaRE

Tables 5 and 6 show the raw values of the OQuaRE metrics for the ontologies developed by the human and by OntoGenix. As described in Section 3.2.2, the range of the values of the metrics may differ, so OQuaRE scales the values to 1–5 scores according to best practices. It should be noted that the column B of Tables 5 and 6 describes whether a high raw value (Highest) or low raw value (Lowest) of the metric corresponds to a high quality score in the range 1–5. Table 7 shows the scaled view of the scores of the metrics, that is, the quality scores. Provided that the 1–5 scale is discrete, the raw values of the metrics exhibit more differences than the quality scores. A combined analysis of the differences for both the raw values and the quality scores of the metric reveals that the OntoGenix ontologies have higher quality scores than the human ones in more metrics for the Brazilian and E-commerce datasets,

Table 6
Raw values of the human (H) and OntoGenix (O) ontologies for the 19 OQuaRE metrics for the Brazilian, Customer and E-commerce datasets. The column Best means whether the highest or the lowest raw value of the metric means the highest quality score. Numbers in bold mean better quality values for the OntoGenix ontologies. The numbers of the Best value row stands for the number of metrics for which a better value was achieved by the human or OntoGenix ontology.

METRIC	Best	BRAZILIAN		CUSTOMER		ECOMMERCE	
		H	O	H	O	H	O
ANOnto	Highest	0.82	**1.86**	1.60	**2.10**	0.56	**3.17**
AROnto	Highest	0.00	**1.86**	2.30	1.17	1.11	1.08
CBOOnto	Lowest	0.91	**0.86**	0.90	1.14	0.89	1.00
CBOOnto2	Lowest	0.91	**0.86**	0.90	1.14	0.89	1.00
CROnto	Highest	0.00	**0.86**	0.00	**0.62**	1.00	**1.75**
DITOnto	Lowest	2.00	2.00	3.00	3.00	3.00	3.00
INROnto	Highest	0.91	0.86	0.90	**1.14**	0.89	**1.00**
LCOMOnto	Lowest	1.00	1.00	1.29	1.44	1.14	1.63
NACOnto	Lowest	1.00	1.00	1.00	1.29	1.00	1.14
NOCOnto	Lowest	10.00	**6.00**	3.00	3.43	4.00	**2.40**
NOMOnto	Lowest	10.18	**3.29**	5.50	**4.48**	8.56	**3.42**
POnto	Highest	0.00	**0.57**	0.20	**0.38**	0.11	**0.42**
PROnto	Highest	0.92	0.79	0.86	0.80	0.91	0.77
RFCOnto	Lowest	11.09	**4.14**	6.40	**5.62**	9.44	**4.42**
RROnto	Highest	0.08	**0.21**	0.14	**0.20**	0.09	**0.23**
TMOnto	Lowest	0.00	0.00	0.00	0.00	0.00	0.00
TMOnto2	Highest	0.00	0.00	0.00	0.00	0.00	0.00
WMCOnto	Lowest	1.00	1.00	1.29	1.86	1.14	1.86
WMCOnto2	Highest	1.00	1.00	1.00	**1.29**	1.00	**1.14**
	Best value	**2**	**10**	**8**	**8**	**7**	**9**

whereas the human one gets higher scores in more metrics for the Airlines and Amazon datasets. The results for the BigBasket and Customer datasets show that both ontologies exhibit better quality scores than the other ones for the same number of metrics, although the raw values are slightly better for the human ontologies. The rest of our analysis will be carried out at the level of quality scores. Table 8 summarizes the differences in terms of quality scores of the metrics, that is, using the scaled values. Their analysis reveals the following major findings:

- The ontologies developed by OntoGenix obtain higher scores for RFCOnto (number of usages of object, datatype properties and superclasses), NOMOnto (average number of properties per class), RROnto (ratio of taxonomic relations and object properties) and ANOnto (ratio of annotations). These metrics are mainly related with the richness in content.
- The human ontologies obtain higher scores for WMCOnto2 (number of paths in the ontology), DITOnto (depth of the hierarchy) and PROnto (ratio of properties). These metrics are mainly related to the complexity of the structure of the ontology.
- The human ontologies developed by A tend to have more metrics with higher scores than the ontologies developed by OntoGenix, whereas the opposite happens for B.

Table 8 shows that there are punctual differences for other metrics. It is worthy noting that NOCOnto gets a higher score for 3 OntoGenix ontologies and for 2 human ones. This metric is related to the number of subclasses and the number of non-leaf classes, thus also related to the complexity of the structure of the ontology. As mentioned, the metrics scores are in the range 1–5. If we only consider those metrics whose values differ in at least 2 units in the scale (for instance 5 and 3), the ontologies developed by OntoGenix would have higher scores for the following metrics: ANOnto (BigBasket, Customer, E-commerce), RFCOnto(Brazilian, E-commerce), AROnto (Brazilian, Customer), NOMOnto (Brazilian), NOCOnto (Brazilian). Consequently, the most significant differences are found for the Brazilian and E-commerce datasets. These are domains modeled by ontologist B. On the contrary, 2 units in the scale higher scores would be obtained by the human ontologies for WMCOnto (Airlines), TMOnto2 (Amazon) and ANOnto (Amazon). Airlines and Amazon are domains modeled by ontologist A.

Table 8 summarizes, by dataset, for which metrics the score obtained by the OntoGenix ontology (column OntoGenix) was different from the human one (column Human). The number in brackets associated with each dataset represents the number of OQuaRE metrics that obtained the same score for both ontologies for the corresponding dataset. For example, both ontologies had the same quality score for 13 metrics for the Airlines dataset, this is why six OQuaRE metrics are included in the OntoGenix and Human columns. We can see that, for the airlines dataset, RFCOnto and NOMOnto appear in the OntoGenix column but not in the Human one. This means that the scores obtained by the OntoGenix ontology for those metrics was higher than for the human one. On the contrary, the human ontology obtained a higher score for WMCOnto2, NOCOnto, DITOnto and LCOMOnto. The last column (Who) shows which human ontologist developed the human ontology.

4.3.2. Quantitative assessment with OOPS!

Tables 9 and 10 describe the pitfalls found in the ontologies developed by humans and by OntoGenix respectively. The OntoGenix ontologies accumulated 59 occurrences of 16 pitfalls, whereas the human ontologies exhibited 38 occurrences of 13 pitfalls. The

Table 7
Scaled values of the human (H) and OntoGenix (O) ontologies for the 19 OQuaRE metrics, where 1 is the lowest score and 5 is the highest one. Air= Airlines; Ama= Amazon; Big= BigBasket; Bra= Brazilian; Cus= Customer; E-c= E-commerce. The numbers of the Best value row stands for the number of metrics for which a higher score was achieved by the human or OntoGenix ontology.

METRIC	Air		Ama		Big		Bra		Cus		E-c	
	H	O	H	O	H	O	H	O	H	O	H	O
ANOnto	5	5	4	1	2	5	5	5	5	5	3	5
AROnto	5	5	5	5	5	5	1	5	5	5	5	5
CBOOnto	5	5	5	5	5	5	5	5	5	5	5	5
CBOOnto2	5	5	5	5	5	5	5	5	5	5	5	5
CROnto	5	5	1	1	1	1	1	5	1	4	5	5
DITOnto	5	4	5	4	5	5	5	5	4	4	4	4
INROnto	5	5	4	5	4	4	5	5	5	5	5	5
LCOMOnto	5	4	5	5	5	5	5	5	5	5	5	5
NACOnto	5	5	5	5	5	5	5	5	5	5	5	5
NOCOnto	3	2	5	5	5	4	2	4	5	4	4	5
NOMOnto	4	5	3	4	1	1	1	4	3	3	1	4
POnto	5	5	5	5	5	5	5	5	5	5	5	5
PROnto	4	4	5	4	5	5	5	4	5	4	5	4
RFCOnto	4	5	4	4	1	1	2	4	3	4	2	4
RROnto	2	2	1	2	1	1	1	2	1	2	1	2
TMOnto	5	5	5	4	5	5	5	5	5	5	5	5
TMOnto2	5	5	5	3	5	5	5	5	5	5	5	5
WMCOnto	5	5	5	5	5	5	5	5	5	5	5	5
WMCOnto2	5	3	5	4	5	5	5	5	5	4	5	5
Best value	**4**	**2**	**6**	**3**	**1**	**1**	**1**	**6**	**3**	**3**	**1**	**5**

Table 8
Metrics for which different scores are obtained by OntoGenix and the human ontology. The columns represent the dataset (Dataset), the ontologies developed by OntoGenix (OntoGenix and the human (Human), and which ontologist developed the human ontology (Who.

Dataset	OntoGenix	Human	Who
Airlines (13)	RFCOnto, NOMOnto	WMCOnto2, NOCOnto, DITOnto, LCOMOnto	A
Amazon (10)	RROnto, NOMOnto, INROnto	WMCOnto2, TMOnto2, TMOnto, PROnto, DITOnto, ANOnto	A
BigBasket (17)	ANOnto	NOCOnto	B
Brazilian (12)	RROnto, RFCOnto, NOMOnto, NOCOnto, CROnto, AROnto	PROnto	B
Customer (12)	NOCOnto,CROnto, AROnto, ANOnto	WMCOnto2, NOMOnto, DITOnto	A
E-commerce (13)	RROnto, RFCOnto, NOMOnto, NOCOnto, ANOnto	PROnto	B

Table 9
Pitfalls found in the ontologies developed by humans. Air= Airlines; Ama= Amazon; Big= BigBasket; Bra= Brazilian; Cus= Customer; E-c= E-commerce.

Pitfall	Type	Air	Ama	Big	Bra	Cus	E-c	Total
P08	Minor		1	1	1	1	1	5
P10	Important		1		1	1		3
P11	Important			1	1	1	1	4
P13	Minor		1		1	1	1	4
P19	Critical					1		1
P21	Minor					1		1
P22	Minor		1	1	1		1	4
P31	Critical			1				1
P32	Minor				1			1
P36	Minor	1	1	1	1	1	1	6
P37	Critical	1						1
P38	Important	1	1	1	1	1	1	6
P39	Critical	1						1
Total		**4**	**6**	**6**	**8**	**8**	**6**	**38**

pitfalls P2, P4, P7, P24, P35 and P36 were found only in the OntoGenix ontologies, whereas P31, P32 and P37 were found only in the human ontologies. The pitfalls found are classified in critical (11 OntoGenix/4 human), important (22/13) and minor (26/21). Table 11 provides a summary of the occurrences by dataset. The ontologies developed by OntoGenix have more pitfalls for every dataset.

If we focus on pitfalls related to modeling, the most frequent one is *P8. Missing annotations*, which happens for all the ontologies except for the human one for the airlines dataset. *P10. Missing disjointedness* and *P13. Inverse relations not explicitly declared* are present in all the OntoGenix ontologies. *P31. Defining wrong equivalent classes* is present in 5 OntoGenix ontologies. If we consider those pitfalls with at least two occurrences in the human or OntoGenix ontologies, only *P11. Missing domain or range properties* is

Table 10
Pitfalls found in the ontologies developed by OntoGenix. Air= Airlines; Ama= Amazon; Big= BigBasket; Bra= Brazilian; Cus= Customer; E-c= E-commerce.

Pitfall	Type	Air	Ama	Big	Bra	Cus	E-c	Total
P02	MINOR	1			1	1		3
P04	MINOR	1						1
P07	MINOR	1						1
P08	MINOR	1	1	1	1	1	1	6
P10	IMPORTANT	1	1	1	1	1	1	6
P11	IMPORTANT			1				1
P13	MINOR	1	1	1	1	1	1	6
P19	CRITICAL	1						1
P22	MINOR			1		1	1	3
P24	IMPORTANT					1		1
P31	CRITICAL	1		1	1	1	1	5
P34	IMPORTANT	1			1	1	1	4
P35	IMPORTANT	1	1			1	1	4
P36	MINOR	1	1	1	1	1	1	6
P38	IMPORTANT	1	1	1	1	1	1	6
P39	CRITICAL	1	1		1	1	1	5
Total		**13**	**7**	**8**	**9**	**12**	**10**	**59**

Table 11
Summary of pitfalls by dataset.

	Human	OntoGenix	Ontologist
Airlines	4	13	A
Amazon	6	7	A
BigBasket	6	8	B
Brazilian	8	9	B
Customer	8	12	A
E-commerce	6	10	B

more frequent in the human ontologies (4 vs 1). *P2. Creating synonyms as classes*, *P34. untyped class*, and *P35. untyped properties* are not exhibited by human ontologies but by most OntoGenix ontologies. These findings suggest that OntoGenix is modeling in a consistent way across datasets. If we analyze the results by ontologist, the difference in the mean of pitfalls is 4.66 (ontologist A vs OntoGenix) and 2.33 (ontologist B vs OntoGenix).

4.3.3. Qualitative assessment

The manual assessment of the ontologies reveals that, in general terms, human generated ontologies present the following positive features in comparison with the OntoGenix ontologies:

- Axioms of higher expressivity in class restrictions and properties.
- More accurate linking with entities of external ontologies (e.g. Schema), being more interoperable.
- A more nuanced understanding of OWL modeling: instances vs classes, when to add a subsumption relationship, the need for N-ary relationships, etc.

Additionally, some of the OntoGenix ontologies include hallucinations such as entities modeled as both data and object properties, and inappropriate equivalent relations. Finally, the OntoGenix ontologies are consistently richer in annotations, which constitutes an example of how automation could improve some characteristics of the ontologies generated. Most equivalencies defined between classes in the OntoGenix ontologies are inaccurate, and they usually involved entities reused from Schema. BigBasket Products and Brazilian e-commerce are examples of the OntoGenix ontologies with such shortcoming. Regarding the modeling style, Airlines Customer Satisfaction is a significant example. The human modeled the set of satisfaction items as individuals whereas OntoGenix produced a taxonomy, exhibiting a deeply different modeling style. However, if we look at the metrics shown in Table 8, the number of individuals generated by OntoGenix is higher than in the human ontologies. Most of the individuals in the OntoGenix ontology for Airlines Customer Satisfaction represent the set of valid answers for the customer satisfaction item. For example, it generates the individuals Long Departure Delay, Short Departure Delay, and Medium Departure Delay for the class Departure Time Delay.

4.4. Consistency in the modeling of the classes of interest

We analyzed how OntoGenix identified the classes included in the BASF ontology: SalesProduct, SalesArticle, Indication, SalesOrder, AggregatedSalesTransaction, CustomerAccount, Customer Account, Country, CustomerFeeback, ProblemofComplaint, ProblemSubcategory and InferredCause. Table 12 shows the results of how OntoGenix has identified those classes in the datasets. The first column (Classes) represents the names of the classes of interest; the second column (Datasets) represents the number of datasets in which each class is present; the third column (Identified) represents the number of datasets in which OntoGenix has

Table 12
Analysis of the classes of interest identified by OntoGenix.

Classes	D	I	Terms	Columns
SalesProduct	2	2	sales product, product	ProductName, InvoiceNo StockCode
SalesArticle	4	2	salesArticle	articleNumber, StockCode
Indication	1	0		
SalesOrder	1	1	SalesOrder	ProductID, row
AggregatedSalesTransaction	1	0		
CustomerAccount	2	2	Customer	customer id, customerID
Country	2	2	Country, State	customer state, state
CustomerFeedback	2	2	CustomerFeedback, ConsumerFeedback	Customer complaint narrative, satisfaction
ProblemOfComplaint	1	1	ProblemOfComplaint	Issue
ProblemSubCategory	1	1	ProblemSubCategory	SubIssue
InferredCause	1	1	companyResponse	company public response

Table 13
Results of the measurement of the time spent in the different activities of the two ontology construction processes. Time is measured in minutes. The column "Human modeling" stands for the time spent by the human in creating the ontology from scratch. The column LLMReview includes both the time in generating the OntoGenix ontology and the time spent by the human in reviewing that ontology. The column LLM2Human is the time spent by the human in creating the human ontology from the OntoGenix one. The column Time saved is the difference between Human Modeling and the sum LLMReview + LLM2Human. The column Percentage stands for the percentage of time saved, and the column Effort represents the qualitative assessment of the effort for transforming the OntoGenix ontology in the human one.

Time (min)	Human modeling	LLMReview	LLM2Human	Time saved	%	Effort
Airlines	369.0	192.6	22.0	154.4	41.8	2
Amazon	108.8	51.0	5.0	52.8	48.5	1
BigBasket	236.3	139.2	27.0	70.1	29.7	2
Brazilian	105.0	58.2	12.0	34.8	33.1	2
Customer	318.0	262.8	29.0	26.2	8.2	3
E-commerce	375.0	104.4	52.0	218.6	58.3	3

identified a class that can be considered equivalent to the corresponding class of interest; the fourth column (Terms) represents the name assigned to the class identified; the fifth column (Dataset columns) refers to the name of the column associated with the class of interest in the dataset in which the class has been identified by OntoGenix. The results show that the class of interest has been identified in 13 out of 17 occurrences. For 8 out of 11 classes of interest, the classes has always been extracted by OntoGenix. The analysis of the terms and the dataset columns reveals that OntoGenix is not just naming the classes with the name of the dataset column. However, OntoGenix is not always assigning the same name to the class identified, although they are semantically related (sales_product vs product, or country vs state). In the case of the salesOrder class, the value row for "Dataset columns" means that each row is interpreted as an instance of salesOrder. These results show signs of consistency in the modeling and that OntoGenix has some level of understanding of the data and is not only using the names of the fields to create classes.

4.5. Estimation of effort saving

Table 13 shows the results of the estimation of the time saved for each dataset. The generation of the OntoGenix ontology has reduced the total time for generating the human ontology in all the cases, with savings (in minutes) ranging from 26.2 to 218.6 in absolute terms, and from 8.2% to 58.3% in percentage. Regarding the assessment of the effort needed for transforming the OntoGenix ontology into the human one, in all the cases the effort was evaluated 3 (moderate effort) or lower, the most frequent being low effort.

5. Discussion

5.1. The ontologies

We have applied OntoGenix to a series of datasets, and we have compared the ontologies generated with ontologies developed by humans. The assessment of the ontologies generated by OntoGenix showed that they are not as adequate as the human-developed ones, which are more comprehensive and aligned with the content and structure of the CSV files. The qualitative evaluation of the ontologies revealed that despite OntoGenix ontologies are richer in annotations, and in many cases producing a similar number of classes and properties, they are still behind the human reasoning for modeling complex contexts. These findings are also supported by the results of the quantitative metrics, which show differences in metrics related to annotation properties in favor of the OntoGenix ontologies, but the human ones have higher scores for metrics related to the taxonomy or the number of paths, which deal with the complexity of the structure of the ontology. The scores of some metrics related to the number and usage of properties are also higher for the OntoGenix ontologies, but we do not draw any strong conclusion due to the suboptimality of some design decisions

when qualitatively inspected. The analysis of the pitfalls is similar, suggesting that OntoGenix is modeling in a consistent way, with identified strengths and aspect to improve.

These results show the potential of LLMs for increasing the productivity of ontology engineering and KG production, and that tools such as OntoGenix can accelerate ontology development. Even though the ontologies might still lack the quality to be directly applied in applications, they constitute a good starting point that can save a lot of effort to ontology engineers and domain experts. This assertion is reinforced by the results of the measurement of the time saved by the availability of OntoGenix to the ontologists (8.2%–58.3%).

The comparison of the human and OntoGenix ontologies has revealed an unexpected result, the difference between the modeling carried out by the two human ontologists when compared to OntoGenix. The metrics of the OntoGenix ontologies are generally lower than the ones developed by ontologist A, but higher than the ones developed by ontologist B, since patterns have been identified in the datasets studied. OntoGenix ontologies could then be considered baseline ontologies and the human ones could be evaluated against that standards, which could be useful, for example, in ontology development training.

We need to carry out experiments with datasets of varying complexity and from different domains. In addition to this, OntoGenix is closely tied to the capabilities of GPT-4, so any limitations inherent to GPT-4 or changes in its functioning could directly affect OntoGenix performance. Running similar experiments with other LLMs might also generate information about their ontology design capabilities.

The general conclusions of the assessment of the ontologies generated by OntoGenix are in line with the conclusions presented in Babaei Giglou et al. (2023b), although the experimental approach and the goals of both works are different. Given our focus on modeling rather than on learning, we have generated more detailed insights based on metrics, pitfalls and human judgment which have enabled us to identify both strengths and flaws in the ontologies generated by OntoGenix. The evaluation performed is also larger than the one presented in Lippolis et al. (0000), since we have used more datasets, which has also enabled us to study additional aspects such as the consistency of the modeling of different datasets by using as reference an ontology provided by BASF SE related to the domain of e-commerce.

5.2. The method

We believe that our study on OntoGenix constitutes a step forward in the field of ontology engineering, primarily driven by the integration of LLMs, particularly GPT-4. We found that OntoGenix can accelerate the ontology creation. The OntoGenix potential contribution to increased efficiency in ontology development is especially notable when compared to traditional time-consuming and error-prone methods.

The implications of these findings go beyond just improved efficiency and accuracy. OntoGenix has the potential to democratize the field of ontology engineering, making it accessible even to those without deep expertise in this area.

The modular design of OntoGenix allows for adaptability to various domains and requirements. Each stage is configurable, enabling adjustment and customization to meet specific ontology engineering needs. This modularity and configurability ensures the scalability and versatility of OntoGenix in diverse application scenarios.

One of the most striking innovations of OntoGenix is its utilization of GPT-4. By leveraging such an advanced LLM, OntoGenix benefits from an unparalleled depth of language understanding and generation capabilities. Moreover, the OntoGenix comprehensive workflow, which spans from initial data processing to the final generation of KGs, ensures a seamless and integrated process. This not only saves time, but also ensures a higher standard of accuracy and relevance in the final output.

5.3. Selection of the LLM

In this work we have used OpenAI GPT as the LLM for supporting the creation of the ontologies, but this is not the only LLM available. In the last years different LLMs have been developed, some being open source[7] such as Meta's Llama,[8] and some not being open source, as it is the case of OpenAI GPT or Google's Gemini.[9] When we started this work there was not much information available on the performance of LLMs for ontology development related tasks.

Our selection of OpenAI GPT was guided by the results shown in Babaei Giglou et al. (2023a). There, different LLMs were tested for the ontology learning task. That work had three research questions and for two of them, OpenAI GPT got the best results. We did some initial tests with OpenAI GPT-4 and other open source LLMs for ontology development, and the results were in line with the ones from Babaei Giglou et al. (2023a). The use of OpenAI GPT-4 had a cost for the development of the six OntoGenix ontologies. The number of tokens consumed and the cost of those six ontologies is shown in Table 14. The overall cost is around $0.1 using the cost per token available at the time of writing this article, which does not seem prohibitive for an organization.

The use of open source LLMs would have several theoretical advantages such as cost reduction, non-dependency of the provider, more transparency in the methods and the software code, higher customization options, and more control on the data used. Non-open source LLM vendors offer enterprise options, in which data privacy should not be a concern. The field of LLMs is in continuous evolution, and new versions of existing LLMs and new LLMs are frequently released. A continuous surveillance of the LLMs available

[7] https://github.com/eugeneyan/open-llms

[8] https://llama.meta.com

[9] https://gemini.google.com

Table 14
Summary of the cost for the development of the OntoGenix ontologies.

Dataset	Total tokens	Total cost (USD)
Airlines Customer Satisfaction	8722	0.0218
Amazon Ratings	5433	0.0136
BigBasket Products	6704	0.0168
Brazilian E-commerce	6280	0.0157
Customer Complaint	11 633	0.0291
E-commerce	4408	0.0110

and of their performance is needed, since they offer more tokens, are trained with more parameters, the cost of usage for the non-open source ones gets cheaper, and more APIs become available. Given the evolution of the field, the results obtained using OpenAI GPT-4 can be considered a baseline. Carrying out regular, systematic studies of the performance of OntoGenix with different LLMs would permit adjusting it to use the LLM that provides the best result and, therefore, to improve the results. That would complement other comparative studies of LLMs (Liu, Li et al., 2024; Qi et al., 2023).

5.4. Impact on KG generation and quality

OntoGenix is able to generate ontologies, mappings from the dataset to the ontology generated in RML and RDF data by processing the dataset and the mappings with Morph-KGC (Arenas-Guerrero, Chaves-Fraga, Toledo, Pérez, & Corcho, 2024). Therefore, the work presented here has a clear impact on the generation of KGs and, more specifically, the automated generation of KGs. These processes are driven by the ontology generated, and this is why the effort in this article has been put on the assessment of the ontologies. If we are able to reduce the effort in the generation of ontologies and those ontologies are extracted from the datasets, the cost for generating the KG will also be drastically reduced.

In the process of generating the ontology, the LLM is already linking the content of the data with the classes and properties of the ontology, producing the basic mapping. In this paper, we do not have assessed the quality of the mappings automatically generated, which should be the focus of a different work. The analysis of the automated mappings should also inform about gaps in the modeling of the ontologies, that is, data columns not included in the mapping. The results on the consistency of the modeling is also encouraging from the perspective of KG construction from different datasets. In case the classes generated by OntoGenix were based on the names of the data columns, without showing any sign of understanding, it would mean that a high effort for entity linking would be needed. The signs of consistent modeling have also a positive impact on the quality of the results, since similar principles would be applied. The challenge here turns into being able to constraint the modeling of OntoGenix to the desired design patterns.

5.5. Potential applications of OntoGenix

Our initial application is the increase of the productivity in the development of ontologies and KGs. More and more large companies are using or exploring KGs as their technological solution for supporting data integration. Such companies have many heterogeneous, distributed data sources from different subdomains of their business area. The ontologies of those datasets are usually developed by the ontology engineers of the company, who normally have little or no knowledge about those subdomains. In many cases, the data for the development of the ontologies comes from more than one legacy system of the company, which makes it even harder. They usually need the support from domain experts for the ontology development, especially in the first stage, and this has a high cost for the company. We think that OntoGenix has the potential to partially fill the role of those domain experts, helping the ontology engineers to draft and initial version that allows them to adjust and improve the modeling, obtaining a reduction in the time needed for the development of the ontology, and a reduction of the cost associated with the participation of both the ontology engineers and domain experts. OntoGenix holds promise in a variety of domains and data and knowledge-intensive industries. The rapid development of ontologies would facilitate the development of semantic KGs, which would promote the development and application of neuro-symbolic artificial intelligence approaches. Business intelligence would benefit from having the possibility of the faster creation of KGs. The consistent modeling exhibited by OntoGenix would also contribute to that, since the data integration effort would be reduced. That would also benefit applications in the healthcare domain since patient data is usually distributed over heterogeneous sources. This is an area for which the possibilities of LLMs are being studied (Liu, Yang et al., 2024). It has therefore potential to have a high impact in areas and industries which require huge data integration such as health, agri-food or manufacturing. In the latter case, the manufacturing process goes through several stages for which different systems are used, and a KG connecting all that information would help to optimize the management of the information. In the last years, there has been an increasing interest in the development of KGs in Law (Lahiri, Pai, Weninger, & Bhattacharya, 2024; Shi et al., 2024). Moreover, LLMs specific for Law have been proposed (Shu et al., 2024), to which OntoGenix could be adapted. OntoGenix could significantly contribute to increase the amount of FAIR datasets available, since the data could be expressed in RDF with little effort. That would be important to support initiatives such as the Common European Data Spaces, which includes the European Open Science Cloud.

5.6. Limitations and further work

OntoGenix has also some limitations. A primary concern is scalability, particularly with larger and more complex datasets where its efficacy has yet to be thoroughly evaluated, and strategies for overcoming the token limit of GPT-4 should be developed. Additionally, the tool performance is closely tied to the capabilities of GPT-4. As such, any limitations inherent to GPT-4 or changes in its functioning could directly affect OntoGenix's performance.

The results have shown that OntoGenix saves time in ontology development. It should be noted that the fact that the same ontologists who developed the human ontologies also carried out the assessment of the time saved by OntoGenix may have overestimated in some way the time saved by OntoGenix, given that they were already familiar with the domain classes and properties. However, another ontologist, who was not involved in the development of the OntoGenix, participated in the qualitative assessment of the effort. These results are also quite positive, which limits the aforementioned potential overestimation of the time saving.

Further studies will investigate the application of OntoGenix in real-world scenarios across various industries to validate and expand its utility. By advancing the capabilities of OntoGenix, we aim to foster a semantically richer and accessible web, enabling enhanced data discovery, analytics, and decision-making across the globe.

The future trajectory for OntoGenix is poised to further this advancement by integrating a repository of local ontologies, facilitated by tools like Plow.[10] This integration is anticipated to bolster the collaborative and cumulative aspects of ontology engineering, allowing for the creation of a shared knowledge base that can be leveraged and expanded upon by researchers and practitioners alike.

Maximizing the impact of OntoGenix hinges on enhancing its usability and accessibility. Current and future versions of the software focus on offering extensive documentation and clear examples to guide users.

6. Conclusions

OntoGenix is an LLM-powered pipeline for increasing the automation of ontology development. The results in the commerce data domain show that the ontologies produced exhibit patterns of coherent modeling and closely resemble those created by humans. Our study underscores the transformative potential of OntoGenix in ontology engineering and KG generation. While acknowledging its current limitations, the path forward is ripe with opportunities for further enhancements and applications, hinting at a future where OntoGenix could be an indispensable tool across multiple sectors. Further work will focus on extending the evaluation and improving the pipeline to improve the precision of the modeling.

CRediT authorship contribution statement

Mikel Val-Calvo: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization. **Mikel Egaña Aranguren:** Writing – review & editing, Writing – original draft, Validation, Investigation. **Juan Mulero-Hernández:** Writing – review & editing, Validation, Investigation. **Ginés Almagro-Hernández:** Writing – review & editing, Validation, Investigation. **Prashant Deshmukh:** Writing – review & editing, Validation, Conceptualization. **José Antonio Bernabé-Díaz:** Writing – review & editing, Validation, Conceptualization. **Paola Espinoza-Arias:** Writing – review & editing, Validation, Conceptualization. **José Luis Sánchez-Fernández:** Writing – review & editing, Validation, Conceptualization. **Juergen Mueller:** Supervision, Funding acquisition, Conceptualization. **Jesualdo Tomás Fernández-Breis:** Writing – original draft, Validation, Supervision, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research has been funded by MCIN/AEI/10.13039/501100011033/ [grant numbers PID2020-113723RB-C22] and by BASF SE.

Data availability

The data are available on GitHub as indicated in the paper.

[10] https://plow.pm/

References

Al-Aswadi, F. N., Chan, H. Y., & Gan, K. H. (2020). Automatic ontology construction from text: a review from shallow to deep learning trend. *Artificial Intelligence Review*, *53*, 3901–3928.

Arenas-Guerrero, J., Chaves-Fraga, D., Toledo, J., Pérez, M. S., & Corcho, O. (2024). Morph-KGC: Scalable knowledge graph materialization with mapping partitions. *Semantic Web*, *15*(1), 1–20.

Babaei Giglou, H., D'Souza, J., & Auer, S. (2023a). LLMs4OL: Large language models for ontology learning. In *International semantic web conference* (pp. 408–427). Springer.

Babaei Giglou, H., D'Souza, J., & Auer, S. (2023b). LLMs4ol: Large language models for ontology learning. In T. R. Payne, V. Presutti, G. Qi, M. Poveda-Villalón, G. Stoilos, L. Hollink, Z. Kaoudi, G. Cheng, & J. Li (Eds.), *The semantic web – ISWC 2023* (pp. 408–427). Cham: Springer Nature Switzerland.

Brants, T., Popat, A. C., Xu, P., Och, F. J., & Dean, J. (2007). Large language models in machine translation. https://aclanthology.org/D07-1090.pdf.

Cai, Y., Mao, S., Wu, W., Wang, Z., Liang, Y., Ge, T., et al. (2023). Low-code LLM: Visual Programming over LLMs. ArXiv preprint arXiv:2304.08103.

Caufield, J. H., Hegde, H., Emonet, V., Harris, N. L., Joachimiak, M. P., Matentzoglu, N., et al. (2024). Structured prompt interrogation and recursive extraction of semantics (SPIRES): A method for populating knowledge bases using zero-shot learning. *Bioinformatics*, *40*(3), btae104.

Chung, W., Cahyawijaya, S., Wilie, B., Lovenia, H., & Fung, P. (2023). InstructTODS: Large language models for end-to-end task-oriented dialogue systems. ArXiv preprint arXiv:2310.08885.

Dash, S., Bagchi, S., Mihindukulasooriya, N., & Gliozzo, A. (2023). Linking Tabular Columns to Unseen Ontologies. In *International semantic web conference* (pp. 502–521). Springer.

Demszky, D., Yang, D., Yeager, D. S., Bryan, C. J., Clapper, M., Chandhok, S., et al. (2023). Using large language models in psychology. *Nature Reviews Psychology*, 1–14.

Duque-Ramos, A., Fernández-Breis, J. T., Stevens, R., & Aussenac-Gilles, N. (2011). OQuaRE: A SQuaRE-based approach for evaluating the quality of ontologies. *Journal of Research and Practice in Information Technology*, *43*(2), 159–176.

Fan, Z., & Chen, C. (2024). CuPe-KG: Cultural perspective–based knowledge graph construction of tourism resources via pretrained language models. *Information Processing & Management*, *61*(3), Article 103646.

Fan, T., & Wang, H. (2022). Research of Chinese intangible cultural heritage knowledge graph construction and attribute value extraction with graph attention network. *Information Processing & Management*, *59*(1), Article 102753.

Gao, J., Peng, P., Lu, F., Claramunt, C., Qiu, P., & Xu, Y. (2024). Mining tourist preferences and decision support via tourism-oriented knowledge graph. *Information Processing & Management*, *61*(1), Article 103523.

Gomez-Perez, J. M., Pan, J. Z., Vetere, G., & Wu, H. (2017). Enterprise knowledge graph: An introduction. In *Exploiting linked data and knowledge graphs in large organisations* (pp. 1–14). Springer.

Hatakeyama-Sato, K., Yamane, N., Igarashi, Y., Nabae, Y., & Hayakawa, T. (2023). Prompt engineering of GPT-4 for chemical research: what can/cannot be done? *Science and Technology of Advanced Materials: Methods*, *3*(1), Article 2260300.

Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., Melo, G. D., Gutierrez, C., et al. (2021). Knowledge graphs. *ACM Computing Surveys (Csur)*, *54*(4), 1–37.

Hu, C., Fu, J., Du, C., Luo, S., Zhao, J., & Zhao, H. (2023). ChatDB: Augmenting LLMs with databases as their symbolic memory. ArXiv preprint arXiv:2306.03901.

Jupp, S., Horridge, M., Iannone, L., Klein, J., Owen, S., Schanstra, J., et al. (2012). Populous: a tool for building OWL ontologies from templates. *BMC Bioinformatics*, *13*(1), 1–12.

Kannan, S. S., Venkatesh, V. L., & Min, B.-C. (2023). SMART-LLM: Smart multi-agent robot task planning using large language models. ArXiv preprint arXiv:2309.10062.

Lahiri, S., Pai, S., Weninger, T., & Bhattacharya, S. (2024). Learning from Litigation: Graphs and LLMs for Retrieval and Reasoning in eDiscovery. ArXiv preprint arXiv:2405.19164.

Li, Y., Lin, Z., Zhang, S., Fu, Q., Chen, B., Lou, J.-G., et al. (2023). Making language models better reasoners with step-aware verifier. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 5315–5333).

Li, H., Su, J., Chen, Y., Li, Q., & Zhang, Z.-X. (2024). SheetCopilot: Bringing software productivity to the next level through large language models. *Advances in Neural Information Processing Systems*, *36*.

Lippolis, A. S., Ceriani, M., Zuppiroli, S., & Nuzzolese, A. G. (2024). Ontogenia: Ontology generation with metacognitive prompting in large language models. https://2024.eswc-conferences.org/wp-content/uploads/2024/05/77770251.pdf.

Liu, J., Chabot, Y., Troncy, R., Huynh, V.-P., Labbé, T., & Monnin, P. (2023). From tabular data to knowledge graphs: A survey of semantic table interpretation tasks and methods. *Information Processing & Management*, *76*, Article 100761.

Liu, Y., Li, D., Wang, K., Xiong, Z., Shi, F., Wang, J., et al. (2024). Are LLMs good at structured outputs? A benchmark for evaluating structured output capabilities in LLMs. *Information Processing & Management*, *61*(5), Article 103809.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., et al. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, *12*, 157–173.

Liu, L., Yang, X., Li, F., Chi, C., Shen, Y., Lyu, S., et al. (2024). Towards Automatic Evaluation for LLMs' Clinical Capabilities: Metric, Data, and Algorithm. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 5466–5475).

Lonsdale, H., O'Reilly-Shah, V. N., Padiyath, A., & Simpao, A. F. (2024). Supercharge your academic productivity with generative artificial intelligence. *Journal of Medical Systems*, *48*(1), 73.

Lynn, S. G. (2008). *Automating mini-ontology generation from canonical tables*. Brigham Young University.

Madani, N., & Joseph, K. (2023). Answering questions over knowledge graphs using logic programming along with language models. In *Tiny papers@ ICLR*.

Mateiu, P., & Groza, A. (2023). Ontology engineering with large language models. In *2023 25th international symposium on symbolic and numeric algorithms for scientific computing* (pp. 226–229). IEEE.

Mizoguchi, R., & Bourdeau, J. (2000). Using ontological engineering to overcome common AI-ED problems. *Journal of Artificial Intelligence and Education*, *11*, 107–121.

Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, *381*(6654), 187–192.

Poveda-Villalón, M., Gómez-Pérez, A., & Suárez-Figueroa, M. C. (2014). Oops!(ontology pitfall scanner!): An on-line tool for ontology evaluation. *International Journal on Semantic Web and Information Systems (IJSWIS)*, *10*(2), 7–34.

Qi, S., Cao, Z., Rao, J., Wang, L., Xiao, J., & Wang, X. (2023). What is the limitation of multimodal llms? A deeper look into multimodal llms through prompt probing. *Information Processing & Management*, *60*(6), Article 103510.

Shen, J., Pan, T., Xu, M., Gan, D., & An, B. (2023). A novel DL-based algorithm integrating medical knowledge graph and doctor modeling for Q&A pair matching in OHP. *Information Processing & Management*, *60*(3), Article 103322.

Shi, J., Guo, Q., Liao, Y., Wang, Y., Chen, S., & Liang, S. (2024). Legal-LM: Knowledge graph enhanced large language models for law consulting. In *International conference on intelligent computing* (pp. 175–186). Springer.

Shu, D., Zhao, H., Liu, X., Demeter, D., Du, M., & Zhang, Y. (2024). LawLLM: Law large language model for the US legal system. ArXiv preprint arXiv:2407.21065.

Thoppilan, R., De Freitas, D., Hall, J., Shazeer, N., Kulshreshtha, A., Cheng, H.-T., et al. (2022). Lamda: Language models for dialog applications. ArXiv preprint arXiv:2201.08239.

Tijerino, Y. A., Embley, D. W., Lonsdale, D. W., Ding, Y., & Nagy, G. (2005). Towards ontology generation from tables. *World Wide Web*, *8*, 261–285.

Weaver, W. N., Ruhfel, B. R., Lough, K. J., & Smith, S. A. (2023). Herbarium specimen label transcription reimagined with large language models: Capabilities, productivity, and risks. *American Journal of Botany*, *110*(12), Article e16256.

Weber, T., Brandmaier, M., Schmidt, A., & Mayer, S. (2024). Significant productivity gains through programming with large language models. *Proceedings of the ACM on Human-Computer Interaction*, *8*(EICS), 1–29.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*, 24824–24837.

Xu, Y., Li, T., Yang, Y., Chen, W., & Yue, L. (2024). An adaptive category-aware recommender based on dual knowledge graphs. *Information Processing & Management*, *61*(3), Article 103636.

Zhou, Z., Song, J., Yao, K., Shu, Z., & Ma, L. (2024). Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning. In *2024 IEEE international conference on robotics and automation* (pp. 2081–2088). IEEE.

Zulkipli, Z. Z., Maskat, R., & Teo, N. H. I. (2022). A systematic literature review of automatic ontology construction. *Indonesia Journal of Electrical Engineering and Computer Sciences*, *28*, 878.